

Automatically react to Claude Code actions

What is a hook

Command hooks run on configured events and receive JSON on stdin. Other hook types include prompt, agent, HTTP and MCP tool hooks; availability depends on the event.

Main events

Event	Moment	Can block
SessionStart	Startup or resume	No
SessionEnd	Session ends	No
UserPromptSubmit	Before prompt processing	Yes
PreToolUse	Before a tool call	Yes
PostToolUse	After a tool succeeds	No
PreCompact	Before context compaction	Yes
Notification	Claude sends a notification	No
Stop	Claude finishes responding	Yes
SubagentStop	A sub-agent ends	Yes

Multi-agent and configuration events

Event	Since	Use
TeammateIdle	v2.1.32+ (Agent Teams)	A team member is about to go idle
TaskCompleted	v2.1.32+ (Agent Teams)	A task is about to be marked complete
ConfigChange	v2.1.49+	A config file changed during the session

Team gates can request further work before completion. `ConfigChange` audits configuration independently of Agent Teams.

Configuration in settings.json

```
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "Bash",
      "hooks": [{
        "type": "command",
        "command":
"${CLAUDE_PROJECT_DIR}/.claude/hooks/
security.sh",
        "args": [], "timeout": 10
      }]
    }]
  }
}
```

Configuration fields

Field	Usage
command	Executable when <code>args</code> is present; otherwise shell command
args	Arguments passed to <code>command</code> without a shell; does not replace the executable
timeout	Seconds; defaults: command 600, prompt 30, agent 60
async	If <code>true</code> , Claude continues without waiting for completion

Field	Usage
<code>continueOnBlock</code>	Prompt hooks: continue after <code>ok: false</code> on supported events, including <code>PreToolUse</code> and <code>PostToolUse</code>

Exit codes

Code	Meaning
0	Success; valid JSON may still deny an action
2	Blocking on supported events; behavior varies by event
Other non-0	Usually a nonblocking error without decision JSON

For `PreToolUse` hooks, exit 2 prevents the tool from executing. Claude reads the `stderr` message and can adapt its behavior.

Synchronous vs asynchronous

Keep validation synchronous. `async: true` runs a command hook in the background and cannot stop the action; use it for notifications or logging. Formatting in the background can race with subsequent edits.

stdin data flow

```
{
  "tool_name": "Edit",
  "tool_input": { "file_path": "src/auth.ts" },
  "tool_response": { "success": true },
  "hook_event_name": "PostToolUse"
}
```

`Stop` includes `last_assistant_message`. Full decisions and exceptions: [Hooks reference](#).